

Recorrido A (por filas):

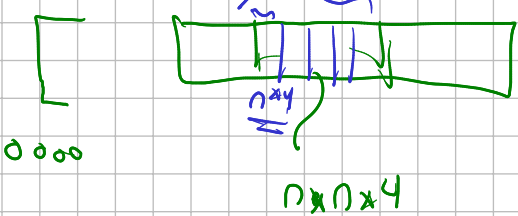
```
for (int i = 0; i < n; i++)
  for (int j = 0; j < n; j++)
    arr[i][j] = i + j;
```

Recorrido B (por columnas):

```
for (int j = 0; j < n; j++)
  for (int i = 0; i < n; i++)
    arr[i][j] = i + j;
```

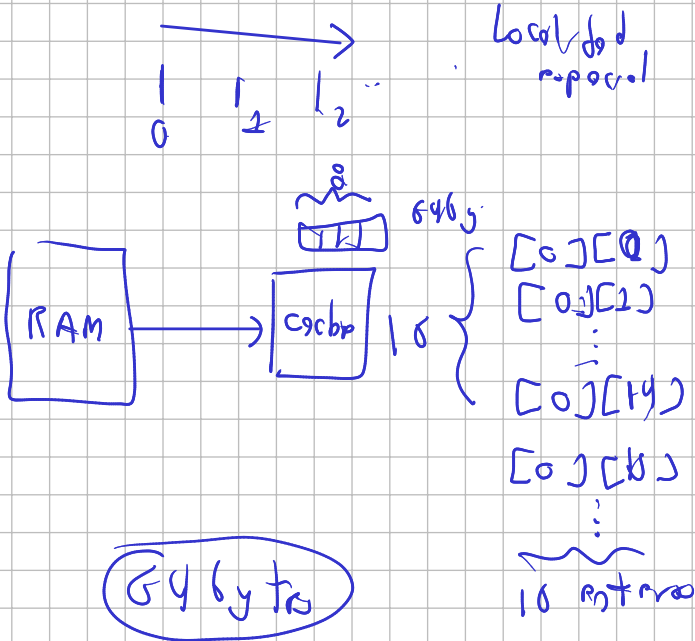
int arr[n][n]

int = 4 bytes = 32 bits

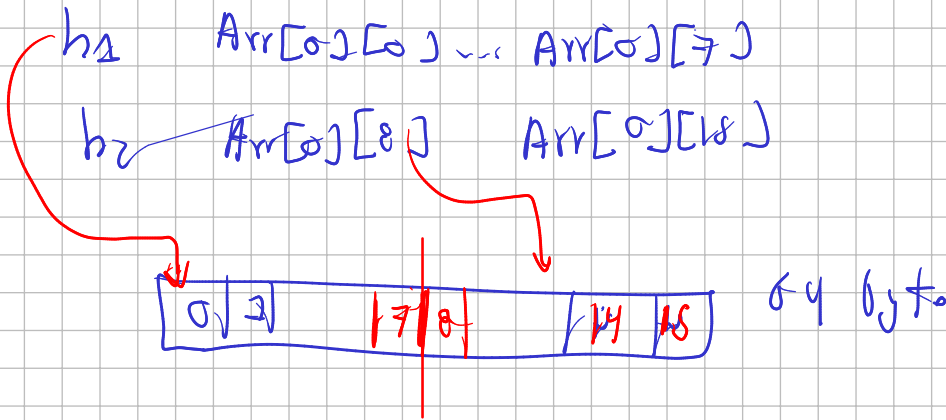
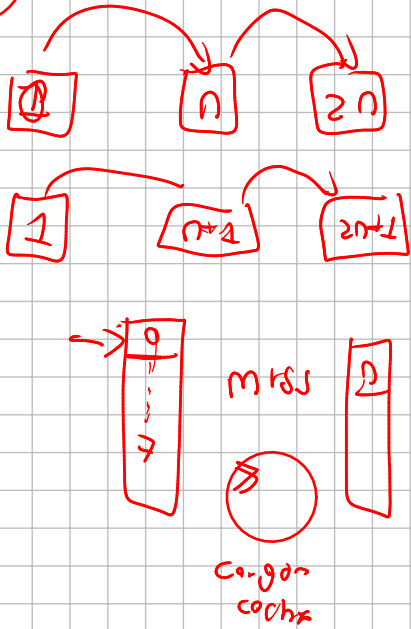


16 GB

A



B



```
double producto_punto(double* a, double* b, int n) {
  double c = 0;
  #pragma omp parallel for reduction(+:c)
  for (int i = 0; i < n; i++)
    c += a[i] * b[i];
  return c;
}
```

copiar por hilo
reducir los copio

OpenMP paralelización, es una libreria que solamente con #pragma {} se puede paralelizar codigo directamente
Nos permite tener una gran libertad para la configuración de la parelizacion

- Codigo en general #pragma omp parallel {...}
- Ciclos for sin condiciones de carrera #pragma omp parallel for
- Condiciones de carrera #pragma omp parallel for reduction(<op>:<variable>)
- Modelos de memoria
- #pragma omp parallel shared(v1,v2,vn) <-- Todos comparten
- #pragma omp parallel private(v1,v2,vn) <-- Cada uno tiene su copia no inicializada
- #pragma omp parallel firstprivate(v1,v2,..vn) <-- Cada uno tiene su copia inicializada

Manejo de ateads

- #pragma omp parallel sections {...} <-- Vamos a colocar varias tareas
- #pragma omp section {} <-- Indica una sección paralela

Barreras

```
#pragma omp barrier <-- Todos los hilos esperan a terminarse
#pragma omp single <-- Solo un hilo cualquiera ejecuta esa seccion
#pragma omp master <-- Solo el hilo principal
```

Condiciones de carrera

```
#pragma omp critical <-- Seccion que solo puede tocar un hilo a la vez
#pragma omp atomic <-- Una operación que se puede hacer al la vez (incrementos, sma,etc)
```

Schedule

```
#pragma omp parallel for schedule(<modo>
```

Modos
static: Todos iguales
dynamic, x <-- >Bajo demanda, tareas desiguales y x hilos al tiempo
guided : como dynamic con tareas dificiles primero
runtime <-- De acuerdo a la configuración del sistema

Los casos de regiones paralelas, como es el caso de un doble for

```
#pragma omp parallel for collapse(2)
```

Va tomar una anidamiento de 2 y lo trabaja como si fuera un solo.

Compilación, detectar errores de OpenMP

```
g++ -std=c++17 -fopenmp -O2 -Wall producto.cpp -o producto
```

incluir openmp

-Wextra

warnings